TALLER · MANTENIMIENTO DE SOFTWARE

Manteni

Control de mantenimiento · Informe de trabajo individual

ESTUDIANTE	DATOS
Nombre	Maikol Lozano
Correo informado	maikollozano6@gmail.com
IdBanner	may180
Repositorio	https://github.com/may180/actividad-mantenimiento-softwar

Resumen

Se presenta Manteni, un prototipo web para consultar activos y órdenes de mantenimiento. El trabajo organiza cinco historias de usuario, implementa una interfaz estática con interacción en memoria y documenta el flujo de control de versiones usado en Git y GitHub.

Alcance de la modalidad

El enunciado original plantea una actividad grupal de hasta tres estudiantes. Este informe se estructura de forma individual a solicitud del estudiante; la aceptación de esta adaptación debe confirmarse con el docente-tutor. El historial y las capturas incluidos corresponden a evidencia real disponible en el repositorio, sin atribuir acciones no verificadas.

Fecha de elaboración: 12/11/2026

1. Producto e historias de usuario

Manteni organiza el trabajo de mantenimiento mediante un resumen de indicadores, inventario de activos, órdenes recientes y formularios de demostración.

ID	Objetivo	Criterio de aceptación
HU-01	Consultar activos	El panel muestra nombre, ubicación y estado para planificar mantenimiento.
HU-02	Registrar activos	El formulario valida campos obligatorios y agrega el activo a la lista.
HU-03	Crear órdenes	La orden muestra prioridad, activo asociado y estado inicial abierto.
HU-04	Priorizar tareas	La tabla distingue prioridad y estado; el indicador cuenta órdenes abiertas.
HU-05	Revisar mi trabajo	La interfaz presenta cinco historias y enlaza al repositorio individual.

Implementación y límites

La aplicación está construida con HTML, CSS y JavaScript sin dependencias de ejecución. Los activos y órdenes de ejemplo se manejan en memoria del navegador y se pierden al recargar; por tanto, la entrega es un prototipo funcional para el taller, no un sistema con persistencia ni una plataforma de producción.

2. Flujo de trabajo con Git y GitHub

El repositorio remoto permite consultar el proyecto y el historial publicado. Los comandos siguientes resumen el flujo de trabajo documentado; los valores entre corchetes se reemplazan según la rama o el archivo trabajado.

Comando	Ejemplo	Propósito
clone	`git clone https://github.com/may180/actividad-mantenimiento-softwar.git`	Obtener una copia local del repositorio.
status	`git status`	Revisar cambios pendientes.
branch	`git switch -c mi-rama`	Crear y cambiar a una rama de trabajo.
add	`git add [archivo]`	Preparar cambios seleccionados.
commit	`git commit -m "Describe el cambio"`	Registrar una unidad de trabajo.
remote	`git remote -v`	Consultar la dirección del repositorio remoto.
push	`git push -u origin mi-rama`	Publicar la rama en GitHub.
log	`git log --oneline --decorate`	Consultar el historial de commits.
ignore	`Agregar archivos locales a .gitignore`	Evitar versionar archivos generados o privados.

Historial visible en el checkout al generar este informe

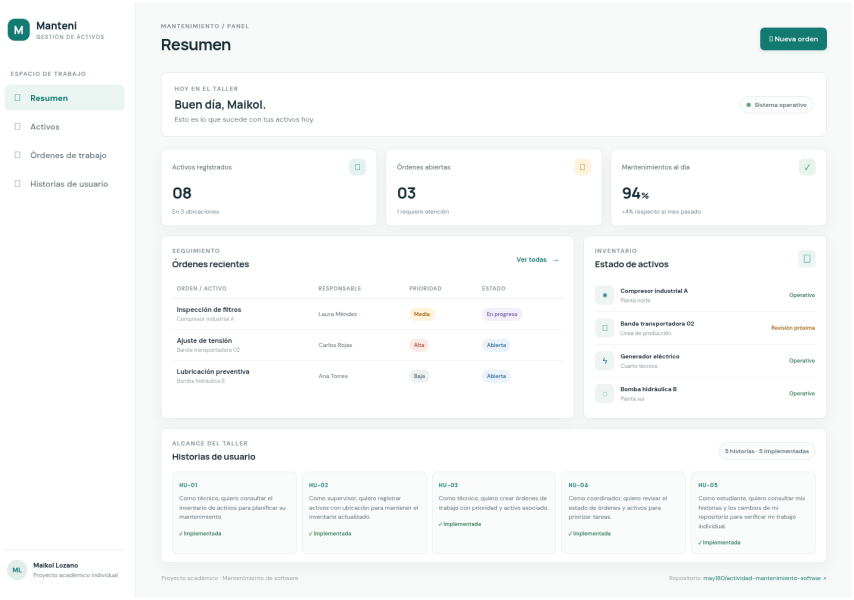
bfff50e | Maikol Lozano | 2026-11-12 | Adapt Manteni workshop for individual submission

e93d3e7 | may180 | 2026-11-11 | Add initial README.md for project

Rama de trabajo: v0/taller-mantenimiento · Repositorio: https://github.com/may180/actividad-mantenimiento-softwar

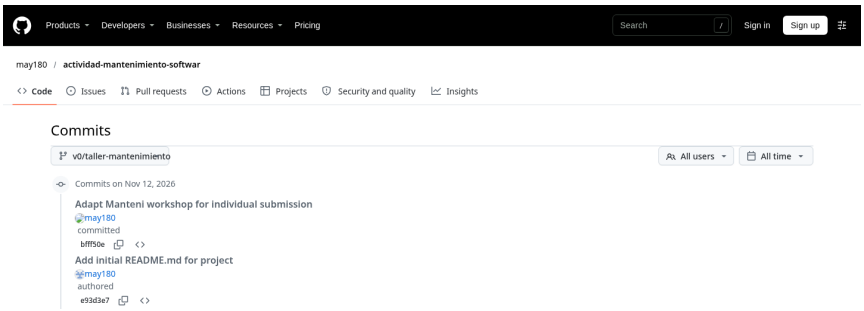
3. Evidencias

Figura 1. Interfaz del prototipo Manteni ejecutándose en navegador.



Fuente: captura del prototipo local; los registros visibles son datos de demostración.

Figura 2. Historial de commits publicado en GitHub.



Fuente: página pública de commits del repositorio y rama de trabajo.

Referencias

Chacon, S., & Straub, B. (2021). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

GitHub. (s. f.). *Acerca de GitHub y Git*. https://docs.github.com/es/get-started/start-your-journey/about-github-and-git